

Approach 3 – Middleware-Based Re-Authentication Architecture

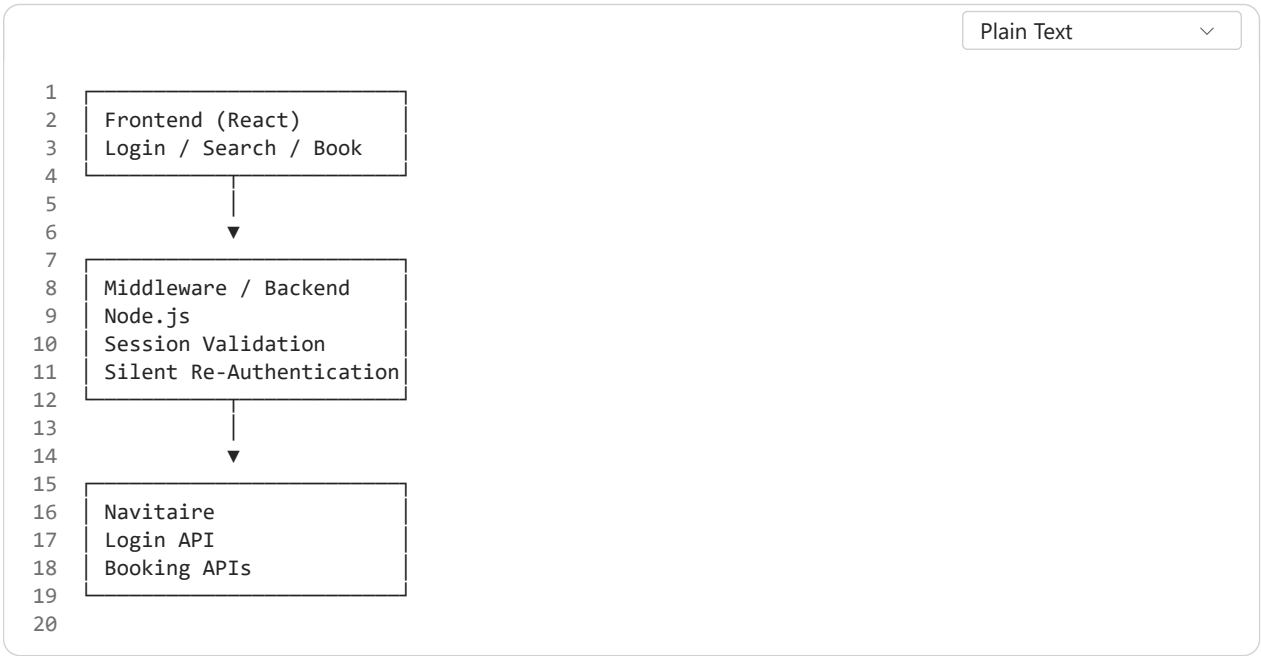
Approach 3 – Middleware-Based Re-Authentication

Complete Stage-Wise Execution Document (Recommended Design)

Objective

Provide persistent login for up to **12 hours** while complying with the Navitaire limitation that authentication tokens expire after **15 minutes** and must be regenerated using the Login API.

1. Final Architecture



2. Components and Responsibilities

Frontend

Responsible For

- Login Screen
- OTP Collection
- Remember Me Checkbox

- Collect Browser Information
- Call Backend APIs
- Handle Session Timeout

Never Responsible For

- Storing Password
- Storing Credentials
- Calling Navitaire Directly
- Generating Navitaire Tokens
- Decrypting Credentials

Backend / Middleware

Responsible For

- Authenticate User
- Generate Browser Hash
- Store Session Data
- Generate Middleware JWT
- Validate Sessions
- Validate Browser Fingerprint
- Silent Re-Authentication
- Renew Navitaire Tokens
- Manage 12-Hour Login Window

Navitaire

Responsible For

- Authentication
- Token Generation
- Booking APIs

Limitation

- One Token Model
- 15 Minute Token Validity
- No Refresh Token

3. Initial Login Flow

User Inputs

JSON



```
1 {
2   "username": "john",
3   "password": "xxxxx",
4   "otp": "123456",
5   "persistLogin": true
6 }
7
```

Browser Parameters

JSON

```
1 {
2   "userAgent": "Mozilla/5.0",
3   "acceptLanguage": "en-US",
4   "screenWidth": 1920,
5   "screenHeight": 1080,
6   "timezone": "Asia/Dubai"
7 }
8
```

Frontend Request

HTTP

```
1 POST /login
2
```

JSON

```
1 {
2   "username": "john",
3   "password": "xxxxx",
4   "otp": "123456",
5   "persistLogin": true,
6   "screenWidth": 1920,
7   "screenHeight": 1080,
8   "timezone": "Asia/Dubai"
9 }
10
```

4. Backend Processing During Login

Step 1 – Receive Request

Backend receives:

- Username
- Password
- OTP

- Persist Login Flag
- Browser Information

Step 2 – Generate Browser Fingerprint

JavaScript

```
1 fingerprint =  
2 userAgent +  
3 acceptLanguage +  
4 screenWidth +  
5 screenHeight +  
6 timezone  
7
```

Example:

Plain Text

```
1 Mozilla/5.0|en-US|1920|1080|Asia/Dubai  
2
```

Step 3 – Generate Browser Hash

JavaScript

```
1 browserHash = SHA256(fingerprint)  
2
```

Step 4 – Call Navitaire Login API

HTTP

```
1 POST Navitaire Login API  
2
```

Response:

JSON

```
1 {  
2   "navitaireToken": "ABC123",  
3   "expiresIn": "15 mins"  
4 }  
5
```

Step 5 – Generate Session

JSON

```
1 {  
2   "sessionId": "S-987654321",  
3   "userId": "123"
```

```
4 }  
5
```

Step 6 – Store Session Record

JSON

```
1 {  
2   "sessionId": "S-987654321",  
3   "userId": "123",  
4   "browserHash": "7ab45f98882fxxx",  
5   "screenWidth": 1920,  
6   "screenHeight": 1080,  
7   "timezone": "Asia/Dubai",  
8   "navitaireToken": "ABC123",  
9   "persistentExpiry": "22:00",  
10  "encryptedCredentials": "***",  
11  "status": "ACTIVE"  
12 }  
13
```

Step 7 – Generate Middleware JWT

Recommended JWT Payload:

JSON

```
1 {  
2   "sessionId": "S-987654321",  
3   "userId": "123"  
4 }  
5
```

Do Not Store:

- Browser Hash
- Navitaire Token
- Password

Step 8 – Return Cookie

HTTP

```
1 Set-Cookie:  
2 middlewareToken=JWT  
3 HttpOnly  
4 Secure  
5 SameSite=Strict  
6
```

5. Normal Request Flow (Token Still Valid)

Timeline

Plain Text



- 1 10:00 Login
- 2 10:05 Search
- 3

Backend Validation Flow

1. Read JWT
2. Fetch Session From Redis
3. Generate Current Browser Hash
4. Validate Browser
5. Validate Navitaire Token
6. Continue Request

Result: Request succeeds without re-authentication.

6. Token Expired Scenario

Timeline

Plain Text



- 1 10:00 Login
- 2 10:15 Token Expired
- 3 10:20 User Clicks Search
- 4

Silent Re-Authentication Flow

1. Validate JWT 
2. Validate Browser Hash 
3. Validate Persistent Window 
4. Detect Expired Navitaire Token 
5. Read Encrypted Credentials
6. Decrypt Internally
7. Call Navitaire Login API Again
8. Obtain New Token
9. Update Session Store
10. Continue Original Request

User Experience

- No Login Screen
- No OTP Prompt
- No Redirect
- Fully Transparent

7. Idle User Scenario

Timeline

```
1 10:00 Login
2 10:15 Token Expired
3 10:30 Idle
4 11:00 Idle
5 12:00 Idle
6 13:00 Idle
7
```

Plain Text



Backend Activity

```
1 NO API CALLS
2 NO TOKEN REFRESH
3 NO NAVITAIRE LOGIN
4 NO PROCESSING
5
```

Plain Text



Advantage

Zero unnecessary system load during user inactivity.

8. User Returns After Long Idle

Timeline

```
1 13:05 User Clicks Search
2
```

Plain Text



Execution Flow

- Validate JWT
- Validate Browser Hash
- Validate Session
- Validate Persistent Window
- Detect Expired Navitaire Token
- Perform Silent Login
- Get New Token

- Update Redis Session
- Continue Request

User Experience

Search results are returned immediately without a login prompt.

9. Persistent Login Expired

Timeline

```
1 10:00 Login
2 22:30 User Clicks Search
3
```

Plain Text



Backend Response

Persistent Window Check:

```
1 Current Time = 22:30
2 Persistent Expiry = 22:00
3
```

Plain Text



Result: Expired ❌

Cleanup Actions

- Delete Redis Session
- Delete Browser Hash
- Delete Navitaire Token
- Delete Stored Credentials
- Clear JWT Cookie

Response

```
1 401 Unauthorized
2
```

HTTP



Frontend Action

Redirect user to Login page.

10. Complete Runtime Flow

Plain Text

```
1  USER LOGIN
2      ↓
3  Frontend Sends
4      ↓
5  Username
6  Password
7  OTP
8  Persist Login
9  Browser Parameters
10     ↓
11 Backend
12     ↓
13 Generate Browser Hash
14     ↓
15 Navitaire Login
16     ↓
17 Store Session
18     ↓
19 Generate JWT
20
21 =====
22
23 NORMAL REQUEST
24     ↓
25 JWT Cookie
26     ↓
27 Validate JWT
28     ↓
29 Validate Browser Hash
30     ↓
31 Token Valid
32     ↓
33 Continue Request
34
35 =====
36
37 TOKEN EXPIRED
38     ↓
39 Validate JWT
40     ↓
41 Validate Browser Hash
42     ↓
43 Check Persistent Window
44     ↓
45 Silent Navitaire Login
46     ↓
47 Get New Token
48     ↓
49 Continue Request
50
51 =====
52
53 IDLE USER
54     ↓
55 No Requests
56     ↓
57 No Refresh
58     ↓
59 No Processing
60
61 =====
62
63 AFTER 12 HOURS
64     ↓
```

```
65 Persistent Window Expired
66   ↓
67 Destroy Session
68   ↓
69 Force Login
70
```

Recommended Design Summary

JWT

```
1 sessionId
2 userId
3
```

Plain Text



Redis / Database

```
1 browserHash
2 screenWidth
3 screenHeight
4 timezone
5 navitaireToken
6 encryptedCredentials
7 persistentExpiry
8 lastRefreshTime
9 status
10
```

Plain Text



Backend

- Generate Browser Hash
- Validate Browser
- Silent Re-Authentication

Frontend

- Login
- User Actions

Navitaire

- Authentication Only

Key Outcome

A scalable middleware-based solution that supports a secure 12-hour persistent login experience while complying with Navitaire's 15-minute token expiration limitation.

